

Pruebas de Performance — Módulo Bolsillos y Depósito (FuBanking)

Asignatura	Validación y Verificación de Software
Proyecto	FuBanking — Banco digital
Alcance	Bolsillos (5 funcionalidades) + Depositar dinero (módulo Cuentas)
Fase	Pruebas de performance (tiempos de respuesta con presupuesto)
Rama	tests/bolsillos-regresion-fluent
Fecha	2026-10-06

1. Objetivo

1. Medir el **tiempo de respuesta** de las seis funcionalidades contra un **presupuesto explícito** de mediana (p50) y percentil 95 (p95).
2. Verificar cómo **escala** la consulta con 1.000 bolsillos y cuánto tarda un **recorrido completo** del usuario (5 peticiones encadenadas).
3. Demostrar que la suite **detecta un deterioro real**: si una operación se vuelve lenta, la prueba falla con un mensaje claro.
4. Dejar las pruebas dentro del pipeline de Jenkins, junto con la regresión y las unitarias.

2. Alcance

#	Funcionalidad	Método y ruta	Caso
1	Crear bolsillo	POST /api/v1/pockets	PF-01
2	Consultar bolsillos	GET /api/v1/pockets/account/:accountId	PF-02, PF-07
3	Actualizar bolsillo	PATCH /api/v1/pockets/:pocketId	PF-03
4	Eliminar bolsillo	DELETE /api/v1/pockets/:pocketId	PF-04
5	Transferir entre bolsillos	POST /api/v1/pockets/transfer	PF-05
6	Depositar dinero	POST /api/v1/accounts/:id/deposit	PF-06
—	Recorrido completo de 5 peticiones	crear → consultar → actualizar → transferir → eliminar	PF-08

3. Qué se mide y qué no

Qué se mide. Peticiones HTTP **reales** (puerto efímero en **127.0.0.1**, conexiones keep-alive) contra la misma app Express de la regresión: **authMiddleware** con JWT real, validadores zod, controladores, casos de uso y manejador de errores reales. Solo la persistencia va en memoria. Se mide el **costo del código de la aplicación**: autenticación, validación, lógica y serialización.

Qué no se mide.

- **Latencia de red ni de Supabase.** En producción los tiempos serán mayores; estas pruebas detectan que *el código* no se vuelva lento, no estiman la latencia de punta a punta.
- **Carga concurrente ni estrés.** Las peticiones son secuenciales (ver sección 10).
- **Consumo de CPU y memoria.**

4. Metodología

El arnés (`tests/performance/pocket/support/harness.ts`) funciona así:

1. **listen(app)** levanta la app en un puerto libre y la cierra al terminar cada prueba.
2. **measure(...)** ejecuta la operación **5 veces de calentamiento** (descartadas: compilación JIT, conexiones) y **30 repeticiones medidas** con `performance.now()`. El tiempo incluye recibir la respuesta completa.
3. **Correctitud primero.** Cada respuesta debe traer el código HTTP esperado: una operación rápida pero incorrecta **no aprueba**.
4. **prepare** deja el estado listo **fuera** del tiempo medido (por ejemplo, sembrar el bolsillo que se va a eliminar).
5. **Percentil por rango más cercano** (*nearest-rank*): p50 es la mediana y p95 el valor bajo el cual queda el 95 % de las muestras.
6. **Tope de medición de 15 s.** Si una operación está muy por encima del presupuesto no se sigue midiendo; la prueba falla con el mensaje del presupuesto, no por timeout.

Aprueba si: cero respuestas con código inesperado, $p50 \leq \text{presupuesto}$ y $p95 \leq \text{presupuesto}$. Se usan mediana y p95 y no el promedio porque el promedio lo distorsiona un valor atípico y oculta la cola lenta.

5. Catálogo de casos

ID	Caso	Repeticiones	Código esperado	Presupuesto p50 / p95
PF-01	Crear bolsillo (monto 1)	30	201	100 / 300 ms
PF-02	Consultar bolsillos de la cuenta	30	200	100 / 300 ms
PF-03	Actualizar bolsillo (nombre y monto alternante)	30	200	100 / 300 ms
PF-04	Eliminar bolsillo (se siembra uno por repetición)	30	200	100 / 300 ms
PF-05	Transferir 1 peso entre bolsillos	30	200	100 / 300 ms
PF-06	Depositar \$1.000	30	200	100 / 300 ms
PF-07	Consultar una cuenta con 1.000 bolsillos (~150 KB de JSON)	20	200	150 / 400 ms
PF-08	Recorrido de 5 peticiones encadenadas	15	201, 200, 200, 200, 200	250 / 600 ms

6. Resultados

Corrida normal (`01-ejecucion-normal.txt`), tiempos en milisegundos:

Operación	n	min	media	p50	p95	máx	p50 ≤	p95 ≤	Estado
PF-01 Crear bolsillo	30	1,48	2,06	1,84	3,26	3,49	100	300	✓
PF-02 Consultar bolsillos	30	1,11	1,28	1,28	1,52	1,53	100	300	✓
PF-03 Actualizar bolsillo	30	1,26	1,46	1,44	1,74	1,76	100	300	✓
PF-04 Eliminar bolsillo	30	1,05	1,23	1,21	1,45	1,47	100	300	✓
PF-05 Transferir entre bolsillos	30	1,30	1,65	1,49	3,14	4,51	100	300	✓
PF-06 Depositar dinero	30	1,18	1,57	1,34	2,21	5,41	100	300	✓
PF-07 Consultar con 1.000 bolsillos	20	4,63	5,03	4,78	5,72	8,54	150	400	✓
PF-08 Recorrido de 5 peticiones	15	5,34	6,39	6,11	9,95	9,95	250	600	✓

Repetibilidad. En varias corridas, incluida una con cobertura de código activa (que ralentiza la ejecución), las medianas de una sola petición estuvieron entre **1,2 y 5,5 ms**, la consulta con 1.000 bolsillos entre **4,8 y 9,1 ms** y el recorrido de 5 peticiones entre **6,1 y 12,9 ms**.

Escalabilidad (PF-07). Con 1.000 bolsillos la consulta tarda ≈ 5 ms, frente a $\approx 1,3$ ms con 2: crece con el tamaño de la respuesta, pero muy lejos del presupuesto.

7. Presupuestos y holgura

Los presupuestos son **educativos y deliberadamente holgados**, en el espíritu del presupuesto de Créditos del equipo (*presupuesto explícito + mediana*). La suite corre en cada build de Jenkins, donde el hardware es más lento y la cobertura ralentiza la ejecución; un presupuesto ajustado daría **falsos positivos** y se acabaría ignorando.

Caso	Presupuesto p50	Peor mediana observada	Holgura
PF-01 a PF-06 (una petición)	100 ms	5,5 ms	$\approx 18\times$
PF-07 (1.000 bolsillos)	150 ms	9,1 ms	$\approx 16\times$
PF-08 (recorrido)	250 ms	12,9 ms	$\approx 19\times$

Con esa holgura se detecta un retraso de **cientos de milisegundos** (sección 8). **No** se detecta una degradación de $2\times$ o $3\times$ que quede dentro del presupuesto; para eso haría falta comparar contra una línea base y no contra un tope fijo.

8. Sensibilidad: la suite detecta un deterioro

Se introdujo un retraso artificial en el código de producción (y se revirtió) para comprobar que la suite lo detecta.

Cambio introducido	Pruebas que fallan	Mensaje
<code>UpdatePocket.execute</code> : espera de 400 ms al inicio	PF-03 y PF-08	PF-03 Actualizar bolsillo (PATCH): p50 (ms): expected 412.93 to be at most 100
<code>DepositMoney.execute</code> : espera de 150 ms al inicio	PF-06	PF-06 Depositar dinero (POST): p50 (ms): expected 159.42 to be at most 100

El reporte marca la fila como **FUERA** y las demás operaciones siguen en verde, así que el fallo **señala la operación responsable**. Al revertir, la suite vuelve a 8 de 8. Los logs completos están en `docs/evidencia-performance-bolsillos/03-*.txt` y `04-*.txt`.

Lección del proceso. En la primera versión esos retrasos hacían fallar las pruebas por el **timeout de 5 s de Vitest** (35 repeticiones × 400 ms) y no por el presupuesto: se detectaba el problema, pero con un mensaje poco útil. Se corrigió con un timeout explícito de 120 s y el tope de medición de 15 s, para que falle la aserción de p50/p95.

9. Integración con Jenkins

- La etapa *Backend: install + test + build* ejecuta `npm run test:coverage`. Vitest incluye `src/tests/**/*.test.ts`, así que la suite de performance **se ejecuta sin configuración extra**.
- Verificado localmente como lo hace Jenkins: **70 archivos, 597 pruebas aprobadas + 5 it.fails**, 99,4 % de cobertura de líneas y `tsc` sin errores.
- Visibilidad de la tabla.** Con el reporter por defecto de Vitest, la salida de las pruebas que pasan puede no aparecer. `npm run test:performance:bolsillos` usa `--reporter=verbose`, que sí imprime la tabla: úsalo para la evidencia y la demo.
- Riesgo de inestabilidad.** Las pruebas de tiempo dependen de la máquina. Se mitigó con calentamiento, mediana, p95 y presupuestos holgados. Si un runner cargado diera falsos positivos, la primera medida es ampliar los presupuestos, no desactivar la suite.

10. Limitaciones y trabajo futuro

- Carga concurrente y estrés (k6).** El equipo tiene un script k6 para Créditos (`scripts/performance/loans-load.js`: 20 usuarios virtuales, 30 s, p95 < 500 ms). k6 no está instalado pero Docker sí, así que se puede ejecutar con la imagen `grafana/k6` apuntando a la app en memoria levantada en un puerto. No se incluyó porque esta entrega se limitó a tiempos con presupuesto.

- **Contra el backend real.** Incluiría la latencia de Supabase, pero el depósito modifica saldos reales y la tabla `pockets` no existe (el repositorio cae en silencio a un `Map`, defecto D-07).
- **Hipótesis sin verificar: concurrencia.** `updateBalance(accountId, newBalance)` escribe un valor absoluto calculado a partir de una lectura previa. Con una base de datos real, dos operaciones simultáneas sobre la misma cuenta podrían pisarse (actualización perdida). **No está probado:** con el repositorio en memoria las operaciones no se intercalan. Merecería una prueba con latencia simulada.
- **Línea base.** Comparar contra una medición previa detectaría degradaciones menores que el presupuesto.

11. Cómo ejecutar

```
cd backend
npm run test:performance:bolsillos      # solo performance, con tabla de tiempos
npm run test:regression:bolsillos       # regresión + unitarias de las 6 funcionalidades
npm run test:coverage                   # lo mismo que ejecuta Jenkins (suite completa)
```

Requiere el `.env` del backend (`JWT_SECRET`). No requiere Supabase, Docker ni red.

12. Archivos entregados

Rutas relativas a `backend/src/tests/`.

Archivo	Contenido	Estado
<code>performance/pocket/bolsillos.performance.test.ts</code>	8 casos PF-01 a PF-08	Nuevo
<code>performance/pocket/support/harness.ts</code>	Servidor HTTP efímero, <code>measure</code> , percentiles y reporte	Nuevo
<code>backend/package.json</code>	Script <code>test:performance:bolsillos</code>	Modificado
<code>docs/evidencia-performance-bolsillos/</code>	Corrida normal, con cobertura y con cada retraso inyectado	Nuevo

Reutiliza sin cambios el escenario y la app de la regresión (`regression/pocket/support/scenario.ts` y `helpers/createPocketTestApp.ts`).

13. Guion de sustentación

En 30 segundos. Medí el tiempo de respuesta de las seis funcionalidades por HTTP real contra la app en memoria, con 30 repeticiones, calentamiento, mediana y p95 contra un presupuesto, y verificando que cada respuesta sea correcta. También probé que escala con 1.000 bolsillos y que un recorrido completo cabe en el presupuesto, y demostré que detecta un deterioro inyectando retrasos.

Preguntas probables

- **¿Por qué mediana y p95 y no el promedio?** El promedio lo distorsiona un valor atípico y oculta la cola lenta. La mediana muestra la experiencia típica y el p95 la del peor 5 % de las peticiones.
 - **¿Por qué esos presupuestos?** Son educativos y holgados (más de 15× lo observado) para no dar falsos positivos en Jenkins; aun así detectan retrasos de cientos de milisegundos.
 - **¿Estás midiendo la base de datos?** No: mido el costo del código de la aplicación. La latencia de Supabase queda fuera y está indicada en las limitaciones.
 - **¿Es una prueba de carga?** No, es secuencial. Carga y estrés requieren k6 y quedan como trabajo futuro; el equipo ya tiene un script k6 para Créditos como plantilla.
 - **¿Cómo sabes que detecta problemas?** Inyecté 400 ms en `UpdatePocket` y 150 ms en `DepositMoney`: fallaron PF-03 y PF-08, y PF-06, respectivamente, y solo esas.
 - **¿Qué problema tuvo tu primera versión?** Fallaba por timeout y no por presupuesto. Lo corregí con un timeout explícito y un tope de medición.
 - **¿Qué es el calentamiento?** Las primeras ejecuciones son más lentas (compilación JIT, conexiones); se descartan para no medir el arranque.
-

14. Conclusiones

- Las seis funcionalidades responden con **menos de 6 ms de mediana** en una petición y el recorrido completo en ≈ 6 ms, con holgura de más de 15× frente a su presupuesto.
- La consulta **escala bien**: con 1.000 bolsillos tarda ≈ 5 ms.
- La suite **detecta** retrasos inyectados y señala la operación responsable.
- Corre dentro del pipeline sin configuración adicional; entre regresión, unitarias y performance el backend suma **597 pruebas aprobadas**.
- Carga concurrente, estrés y consumo de recursos quedan identificados como siguiente paso.